ARES: Adaptive Reinforcement-based Endpoint Security — An Offline QR-Code Phishing Detector with Calculus-Based Threshold Analysis

Aryan S Jain
RV University
Bengaluru, Karnataka, India
USN: 1RUA25CSE0087
aryanjainbtech25@ rvu.edu.in

Ashwin Chandrashekar Naik
RV University
Bengaluru, Karnataka, India
USN: 1RUA25CSE0090
ashwinchandrashekarnaik btech25@rvu.edu.in

Ashwin Rajesha Bhat
RV University
Bengaluru, Karnataka, India
USN: 1RUA25CSE0091
ashwinrajeshabhatbtech25@rvu.edu.in

Ashwin Subramaniam
RV University
Bengaluru, Karnataka, India
USN: 1RUA25CSE0092
ashwinsubramaniambtech25@ rvu.edu.in

Final-Year AIML Project Report · CIE-3 | Semester 3 · RV University, Bengaluru, Karnataka, India · Submission date: 1 November 2026 · Artefact trace: baseline.json; calculus_experiment.json; calculus_threshold.py; project source and test artefacts

***Abstract*—ARES is an offline QR-code phishing detection prototype that extracts URL-level lexical and structural features, applies a calibrated XGBoost classifier, and maps the predicted phishing probability to allow, review, or block actions. This report examines a two-threshold decision policy and a calculus-based procedure intended to analyse its thresholds. A hard operational cost combines false-positive, false-negative, and review rates, while a sigmoid-smoothed surrogate makes the threshold objective differentiable. The implementation first searches a feasible threshold grid and then applies projected gradient steps using central finite-difference gradients. On the recorded test split, the baseline thresholds (0.35, 0.70) produce hard cost 0.3998726. The returned thresholds (0.60, 0.75) produce hard cost 0.387223, a reduction of 3.16% relative to that baseline. This comparison requires an important qualification: the grid winner (0.30, 0.45) has hard cost 0.2829383, and the subsequent gradient-descent path increases the smoothed surrogate from 0.2907 to 0.4399. Therefore, the improvement of the returned point over the baseline is not evidence that gradient descent reduced the cost; it is attributable to the grid-search starting point and the reported path must be interpreted cautiously. The returned point is boundary-clipped, is not stationary, and is not claimed to satisfy KKT conditions. The report also documents data, prevalence, and generalisation limitations.**

***Index Terms*—QR-code security, phishing detection, URL classification, XGBoost, isotonic calibration, sigmoid surrogate, projected gradient descent, finite differences, threshold optimization.**

I. INTRODUCTION

QR codes are convenient entry points to websites, but a QR code can encode a URL that leads to a phishing destination. A detector that operates without fetching the destination can provide a quick local warning, although URL-only inspection cannot reveal all behaviours that occur after navigation.

ARES is structured as an offline pipeline: a QR image is decoded to obtain a URL; URL-string features are extracted; a trained and calibrated classifier estimates phishing probability; and a two-threshold policy maps the probability to an action. For probability p, the policy allows when $p < t_r$, sends the URL for review when $t_r \le p < t_b$, and blocks when $p \ge t_b$. The thresholds satisfy box bounds and a minimum separation.

The calculus component is an analysis of the decision thresholds, not a claim that the complete application is a reinforcement-learning system. The feedback component is described as a feedback-driven heuristic policy. The threshold experiment uses a smoothed objective, finite-difference derivatives, grid search, and projected updates. The implementation does not establish that the returned point is a stationary or KKT optimum.

Contributions and scope: (1) an offline QR-to-URL detection pipeline; (2) a calibrated URL classifier; (3) a cost-sensitive two-threshold decision rule; (4) a differentiable surrogate and numerical threshold analysis; and (5) an explicit account of limitations and optimisation behaviour. The report evaluates the recorded artefacts and does not claim production deployment or generalisation beyond the constructed evaluation data.

Terminology note: the project name contains "Reinforcement-based", but the policy evidenced by the implementation is heuristic and does not implement an MDP, Q-learning, or a learned reinforcement-learning policy.

II. MATHEMATICAL FORMULATION

A. Two-threshold decision rule

Let p_i be the classifier probability for sample i, and let t_r and t_b denote the review and block thresholds. The decision is:

$$\begin{aligned} &\text{allow if } p_i < t_r; \\ &\text{review if } t_r \le p_i < t_b; \\ &\text{block if } p_i \ge t_b. \end{aligned} \tag{1}$$

The feasible region used by the implementation is

$$\begin{aligned} &0.15 \le t_r \le 0.60, \\ &0.45 \le t_b \le 0.95, \text{ and} \\ &t_b - t_r \ge 0.15. \end{aligned} \tag{2}$$

The resulting action regions for the threshold pairs discussed in this report are shown in Fig. 1.

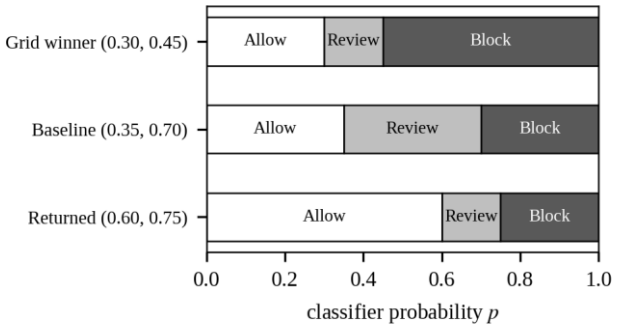


Fig. 1. Allow, review, and block regions on the probability axis for the returned thresholds (0.60, 0.75), the baseline thresholds (0.35, 0.70), and the grid winner (0.30, 0.45).

B. Hard operational cost

The hard rates are

$$\begin{aligned} \mathrm{FPR} &= \frac{fp_review + fp_block}{N_{benign}}, \\ \mathrm{FNR} &= \frac{fn_allow + fn_review}{N_{phish}}, \text{ and} \\ \mathrm{ReviewRate} &= \frac{N_{review}}{N}. \end{aligned} \tag{3}$$

The cost is

$$C = c_{FP} \cdot \mathrm{FPR} + c_{FN} \cdot \mathrm{FNR} + c_{review} \cdot \mathrm{ReviewRate} + R(t_r, t_b), \tag{4}$$

where $c_{FP} = 1$, $c_{FN} = 5$, $c_{review} = 0.5$, and

$$R = 0.05[(t_r - 0.35)^2 + (t_b - 0.70)^2]. \tag{5}$$

The regulariser pulls thresholds toward the baseline defaults (0.35, 0.70). Consequently, this baseline is also the regulariser target; it is not an externally established universal standard.

C. Sigmoid smoothing

For $\tau = 0.05$, define

$$\begin{aligned} \sigma(z) &= 1/(1 + e^{-z}), \\ s_r(p) &= \sigma\big((p - t_r)/\tau\big), \text{ and} \\ s_b(p) &= \sigma\big((p - t_b)/\tau\big). \end{aligned} \tag{6}$$

The surrogate rates are

$$\begin{aligned} \widetilde{\mathrm{FPR}} &= \mathrm{mean}[s_r(p_i) \text{ for } y_i = 0], \\ \widetilde{\mathrm{FNR}} &= 1 - \mathrm{mean}[s_b(p_i) \text{ for } y_i = 1], \text{ and} \\ \widetilde{\mathrm{ReviewRate}} &= \mathrm{mean}\big[s_r(p_i)\big(1 - s_b(p_i)\big)\big] \end{aligned} \tag{7}$$

over all samples. The smoothed objective $\tilde{J}$ uses the same weights and regulariser as C, replacing the hard rates with their smoothed counterparts.

D. Projected gradient update

The implementation uses a fixed learning rate $\eta = 0.05$. It forms $t_r' = t_r - \eta g_r$ and $t_b' = t_b - \eta g_b$, clips each coordinate to its box, and enforces the minimum gap. If the gap is too small, it sets t_b' to $\mathrm{clip}(t_r' + 0.15, 0.45, 0.95)$. The code uses a defensive feasibility guard after this operation.

E. First- and second-order conditions

At an unconstrained interior differentiable minimum, a necessary first-order condition is $\nabla\tilde{J} = 0$. For a strict local minimum, a positive-definite Hessian is a sufficient second-order condition when the usual smoothness assumptions hold. At a constrained boundary point, the unconstrained zero-gradient test is not sufficient; constraint qualification and multiplier conditions would be needed for a KKT analysis. The project artefacts do not verify KKT multipliers. Accordingly, the returned point is described as a boundary-clipped feasible point, not as a verified KKT optimum.

The single-threshold Bayes reference in the JSON is $c_{FP}/(c_{FP} + c_{FN}) = 1/6 \approx 0.1667$. It is a reference for a simplified binary decision setting, not the learned two-threshold solution and not a replacement for the review region.

III. A DIFFERENTIABLE SURROGATE

The hard decision rule changes discontinuously when a threshold crosses a sample probability. Its empirical cost is therefore piecewise constant in the thresholds and does not provide a useful ordinary gradient for gradient-based optimisation. The sigmoid functions replace hard transitions with smooth transitions of temperature τ (Fig. 2). A smaller τ makes the transition sharper; the experiment records $\tau = 0.05$.

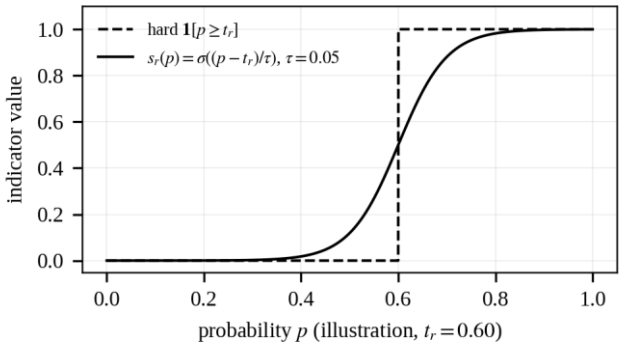


Fig. 2. Illustration of the hard threshold indicator and its sigmoid counterpart $s_r(p)$ for $\tau = 0.05$ (plotted from the stated definition; no experimental data).

The surrogate is an optimisation aid, not the operational cost itself. The final report therefore distinguishes smoothed cost $\tilde{J}$ from hard cost C. This distinction is essential in the experiment: the returned point has smoothed cost 0.439942 and hard cost 0.387223, while the grid winner has a lower hard cost of 0.2829383.

IV. GRADIENT AND OPTIMIZATION ALGORITHM

A. Analytic gradient as a reference derivation

The derivative identity for the sigmoid is $\sigma'(z) = \sigma(z)\big(1 - \sigma(z)\big)$. With $s_r = \sigma\big((p - t_r)/\tau\big)$ and $s_b = \sigma\big((p - t_b)/\tau\big)$, the chain rule gives $\partial s_r/\partial t_r = -s_r(1 - s_r)/\tau$ and $\partial s_b/\partial t_b = -s_b(1 - s_b)/\tau$; the cross derivatives are zero.

For the smoothed rates,

$$\frac{\partial \widetilde{\mathrm{FPR}}}{\partial t_r} = -\frac{1}{N_{benign}\tau}\sum_{y_i=0} s_r\,(1-s_r), \text{ and}$$
$$\frac{\partial \widetilde{\mathrm{FNR}}}{\partial t_b} = \frac{1}{N_{phish}\tau}\sum_{y_i=1} s_b\,(1-s_b). \tag{8}$$

The review-rate derivatives are

$$\frac{\partial \widetilde{\mathrm{ReviewRate}}}{\partial t_r} = -\frac{1}{N\tau}\sum_i s_r\,(1-s_r)(1-s_b), \text{ and}$$
$$\frac{\partial \widetilde{\mathrm{ReviewRate}}}{\partial t_b} = \frac{1}{N\tau}\sum_i s_r\,s_b(1-s_b). \tag{9}$$

Thus, with regulariser coefficient $\lambda = 0.05$ and target $(t_r^0, t_b^0) = (0.35, 0.70)$:

$$\frac{\partial \tilde{J}}{\partial t_r} = -\frac{c_{FP}}{N_{benign}\tau}\sum_{y_i=0} s_r\,(1-s_r) - \frac{c_{review}}{N\tau}\sum_i s_r\,(1-s_r)(1-s_b) + 2\lambda(t_r - t_r^0). \tag{10}$$

$$\frac{\partial \tilde{J}}{\partial t_b} = \frac{c_{FN}}{N_{phish}\tau}\sum_{y_i=1} s_b\,(1-s_b) + \frac{c_{review}}{N\tau}\sum_i s_r\,s_b(1-s_b) + 2\lambda(t_b - t_b^0). \tag{11}$$

This analytic expression is included as a mathematical reference derivation. It is not the gradient routine evaluated by the submitted Python implementation.

B. Implemented numerical gradient

The uploaded `calculus_threshold.py` implements `compute_cost_gradient_fd`. It evaluates the smoothed cost at positive and negative perturbations and uses central finite differences:

$$g_r \approx \frac{\tilde{J}(t_r+\epsilon, t_b) - \tilde{J}(t_r-\epsilon, t_b)}{2\epsilon}, \text{ and}$$
$$g_b \approx \frac{\tilde{J}(t_r, t_b+\epsilon) - \tilde{J}(t_r, t_b-\epsilon)}{2\epsilon}, \tag{12}$$

with $\epsilon = 10^{-4}$. The implementation also estimates the two diagonal Hessian entries and the mixed derivative by finite differences.

C. Recorded returned-point derivatives

TABLE I
RECORDED RETURNED-POINT DERIVATIVES

Quantity	Recorded value	Source field
$\partial \tilde{J}/\partial t_r$	−0.153059	optimized.derivative_check.first_derivative_review
$\partial \tilde{J}/\partial t_b$	1.621676	optimized.derivative_check.first_derivative_block
Stationary / critical point	False	optimized.derivative_check.is_stationary_point
Gradient tolerance	0.01	optimized.derivative_check.grad_tol
Learning rate	0.05	optimized.derivative_check.learning_rate
Finite-difference ϵ	0.0001	calculus_threshold.py, compute_cost_gradient_fd default

The derivative values are nonzero and the JSON marks the point as non-stationary. The threshold coordinates are clipped by the review upper bound and the minimum-gap constraint.

D. One worked update

At the returned point, the recorded review derivative is negative and the block derivative is positive. Before projection, the update direction therefore increases t_r and decreases t_b. The review coordinate cannot increase beyond 0.60, and the minimum-gap constraint prevents the block coordinate from falling below $t_r + 0.15$. The resulting feasible point is (0.60, 0.75), with both the review upper bound and gap constraint active. This illustrates why the final point is boundary-clipped rather than an interior stationary point (Fig. 3).

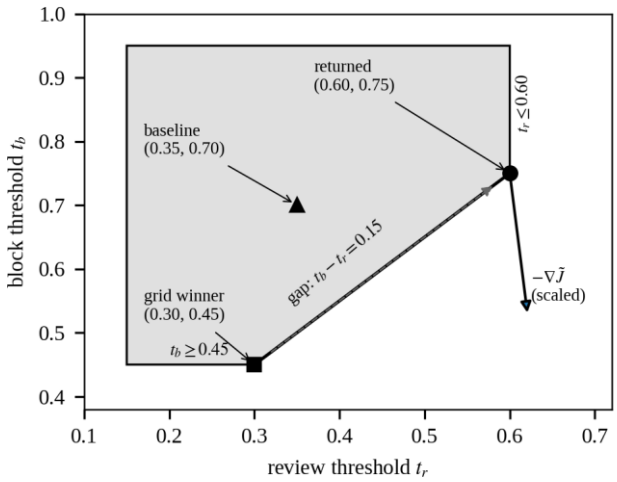


Fig. 3. Feasible region in the (t_r, t_b) plane with the grid winner, baseline, and returned points. The dashed arrow shows only the net displacement between the endpoints (the per-iteration path is not plotted). The solid arrow is the direction of $-\nabla\tilde{J}$ at the returned point, using the recorded gradient (−0.153059, 1.621676), drawn at an arbitrary display scale.

V. OPTIMALITY VERIFICATION

A. First-order assessment

The recorded gradient is (−0.153059, 1.621676), and `is_stationary_point` is false. Hence the unconstrained first-order stationarity condition is not met. The returned point is not described as a stationary optimum.

B. Boundary and KKT qualification

The active constraints recorded are `review_box_0.60` and `gap_0.15`. The derivative signs indicate that the unconstrained descent direction would move the review threshold above its permitted upper bound and the block threshold downward, while the gap constraint prevents that downward movement. KKT multipliers and complementary-slackness conditions have not been verified. No KKT-optimality claim is made.

C. Second-order diagnostic

The finite-difference Hessian at the returned point is

$$H = \begin{bmatrix} 0.405678 & -0.088070 \\ -0.088070 & 17.086192 \end{bmatrix}. \tag{13}$$

Its determinant is 6.923728 and trace is 17.491870. Both diagonal entries are positive and the determinant is positive, so this numerically estimated symmetric 2×2 Hessian is positive

definite at the evaluated point. This is a local curvature diagnostic for the smoothed objective; it does not override the nonzero gradient or establish constrained optimality.

D. Reference threshold

The recorded single-threshold Bayes reference is 0.1667 for the cost weights 1 and 5. ARES uses two thresholds and a review band, so this value is contextual rather than the selected operating threshold.

VI. ARES SYSTEM AND ADAPTIVE POLICIES

The prototype pipeline is: QR image input → local QR decoding → URL string → lexical/structural feature extraction → calibrated XGBoost probability → threshold decision → allow, review, or block (Fig. 4). The application is described as offline with respect to URL inspection; it does not fetch page content or inspect redirect chains as part of the detector.

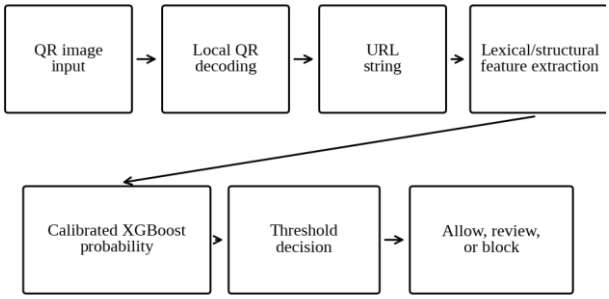


Fig. 4. ARES prototype pipeline.

The feedback-driven heuristic policy adjusts thresholds using a step based on a learning-rate term, a multiplier, and the probability margin. The supplied evidence explicitly does not establish a Markov decision process, Q-learning, or a learned RL policy. The project name should therefore not be used as evidence that reinforcement learning was implemented.

TABLE II
ARES COMPONENTS AND EVIDENCE-BASED DESCRIPTIONS

Component	Role	Evidence-based description
QR decoding	Obtain URL from QR image	Local decoding path; no network inspection
Feature extraction	Represent URL	24 lexical/structural URL features, as stated in supplied project evidence
Classifier	Estimate phishing probability	XGBoost classifier with isotonic calibration
Threshold policy	Map probability to action	Two thresholds: allow / review / block
Feedback adjustment	Adapt policy values	Feedback-driven heuristic, not reinforcement learning
Calculus experiment	Analyse thresholds	Grid search plus projected updates using finite-difference derivatives

VII. EXPERIMENTAL SETUP

The baseline artefact records a GroupShuffleSplit 80/20 split by registrable domain and 5-fold GroupKFold isotonic calibration. The evaluation scope is a held-out domain-grouped evaluation on a semi-synthetic constructed dataset. The test split contains 5,083 URLs from 1,497 domains and has phishing prevalence 0.7478. The baseline JSON reports 17,057 training examples and 5,985 training domains. Random seed is 42.

The classifier is XGBoost (XGBClassifier) with isotonic calibration, migrated from RandomForest. Recorded software versions are scikit-learn 1.9.1, NumPy 2.5.4, and Python 3.14.2. The experiment compares baseline thresholds (0.35, 0.70), a grid winner (0.30, 0.45) documented in the supplied results brief, and the returned thresholds (0.60, 0.75).

Important evaluation qualification: the threshold optimisation uses the same `y_true` and `y_proba` test split that is used to report the threshold rates. No separate held-out threshold-tuning check is present in the final artefacts. Therefore, the threshold comparison is in-sample with respect to threshold selection, despite the classifier's domain-grouped train/test split.

VIII. RESULTS AND ANALYSIS

A. Classifier-level metrics

TABLE III
CLASSIFIER-LEVEL METRICS

Metric	Recorded value	Source
ROC AUC	0.9931	baseline.test.roc_auc
PR AUC	0.9977	baseline.test.pr_auc
Brier score	0.0302	baseline.test.brier
Precision field	0.9833	baseline.test.precision
F1 score	0.9707	baseline.test.f1_score
TPR	0.9584	baseline.test.tpr
FPR	0.0484	baseline.test.fpr
Confusion matrix	[[1220, 62], [158, 3643]]	baseline.test.confusion_matrix

The classifier-level confusion matrix (Fig. 5) is a separate evaluation summary and must not be conflated with the policy-specific outcome counts below. The JSON also records `precision_at_prevalence` values of 0.952 at 50% prevalence, 0.5105 at 5%, 0.1668 at 1%, and 0.0195 at 0.1% (Fig. 6). The supplied limitations note quoted different prevalence-adjusted values; the JSON values are used here as the canonical artefact values.

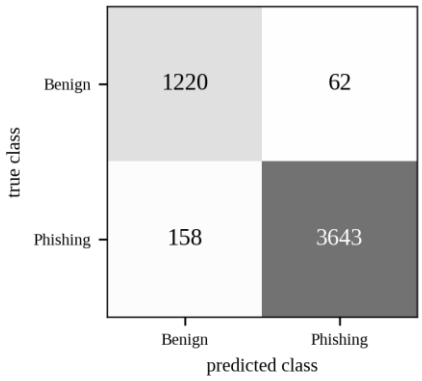


Fig. 5. Classifier-level confusion matrix, [[1220, 62], [158, 3643]] (rows: true class; columns: predicted class).

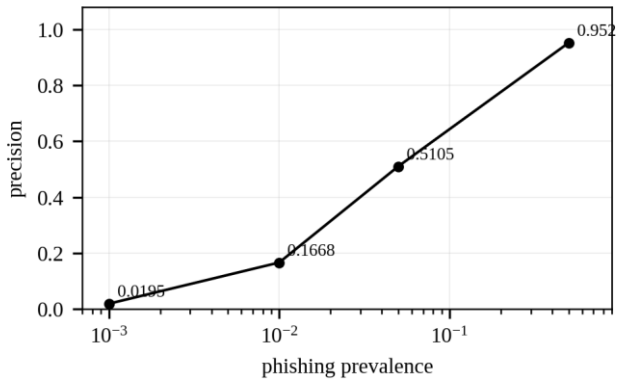


Fig. 6. Recorded precision at phishing prevalence 0.1%, 1%, 5%, and 50%.

B. Threshold rates and hard cost

TABLE IV
THRESHOLD RATES AND HARD COST

Measure	Baseline (0.35, 0.70)	Returned (0.60, 0.75)	Change
FPR	6.94%	3.43%	−50.6% relative
FNR	6.24%	6.81%	+0.57 pp (+9.22% relative)
Review rate	3.74%	1.79%	−52.1% relative
Hard cost	0.3998726	0.387223	−3.16% relative

The returned thresholds reduce the false-positive rate and review rate, but increase the false-negative rate (Fig. 7). The FNR change is +0.57 percentage points; this is not a +0.57% relative change.

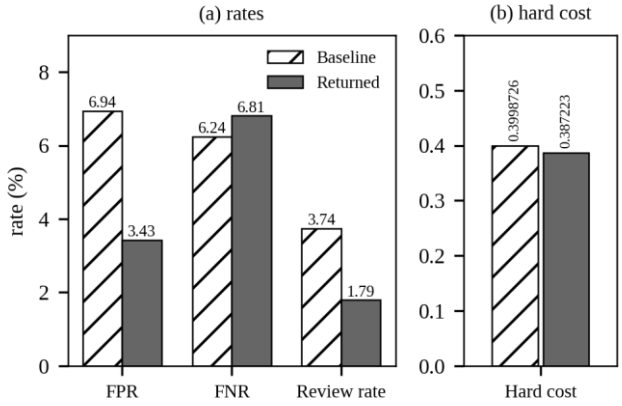


Fig. 7. Baseline (0.35, 0.70) versus returned (0.60, 0.75) thresholds: (a) FPR, FNR, and review rate in percent; (b) hard cost.

C. Policy outcome counts

TABLE V
POLICY OUTCOME COUNTS

Outcome count	Baseline	Returned
fp_review	62	25
fp_block	27	19
tn	1193	1238
fn_allow	109	193
fn_review	128	66
tp	3564	3542

These six outcome categories describe the threshold policy. They differ from the classifier confusion matrix because they separate review and block/allow outcomes (Fig. 8).

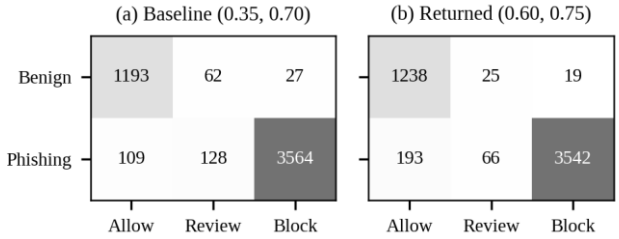


Fig. 8. Policy outcome counts (rows: true class; columns: allow, review, block action) for (a) the baseline and (b) the returned thresholds.

D. Grid search and gradient path

TABLE VI
GRID WINNER, BASELINE, AND RETURNED POINT

Point	Thresholds (t_r, t_b)	Smoothed cost	Hard cost
Grid winner	(0.30, 0.45)	0.2907197	0.2829383
Baseline	(0.35, 0.70)	Not reported	0.3998726
Returned point	(0.60, 0.75)	0.4399421	0.3872226

The history contains 30 entries, corresponding to 29 gradient-descent updates. Along the recorded path, the smoothed surrogate increases from 0.2907197 at the grid winner to 0.4399421 at the returned point (Fig. 9). The returned point's hard cost is also higher than the grid winner's hard cost. The improvement over the baseline is therefore attributable to the grid-search initial point and the subsequent path/selection behaviour, not to gradient descent reducing the objective. This is the central optimisation limitation of the experiment.

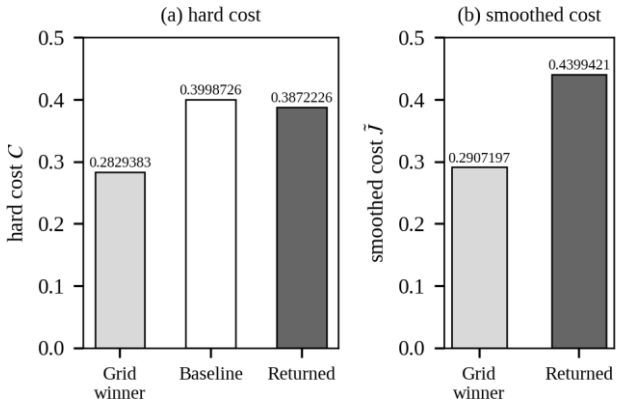


Fig. 9. (a) Hard cost at the grid winner, baseline, and returned point; (b) smoothed cost at the grid winner and returned point (the baseline smoothed cost is not reported).

IX. DISCUSSION AND LIMITATIONS

A. Synthetic benign data

Benign URLs are generated as path and parameter variants of 820 base domains (`train_model.py::benign_variants`). This gives limited benign diversity: variants of a base domain share the base domain's lexical signature, so the model may generalise poorly to real-world benign URLs whose hostnames and paths are not related to the 820 base domains.

B. Lexical features only

Only 24 lexical and structural features are extracted from the URL string. No page content, redirect chain, TLS certificate, or external reputation feed is inspected. The detector therefore cannot catch phishing URLs whose lexical form is benign—for example, a shortened link to a legitimate-looking domain that redirects to a phishing page.

C. Test-set prevalence is not deployment prevalence

The constructed test set has 74.78% phishing prevalence. The baseline JSON records `precision_at_prevalence` values of 0.952 at 50% prevalence, 0.5105 at 5%, 0.1668 at 1%, and 0.0195 at 0.1%. Real-world QR traffic may have a much lower phishing prevalence, so precision can be substantially lower in deployment.

D. In-sample threshold tuning

The reported threshold rates are computed on the 5,083-URL test split, and the calculus thresholds are tuned on the same `y_true` and `y_proba`. No separate held-out threshold-tuning check is present in the final artefacts. A domain-grouped tuning/evaluation split would provide a more independent estimate; the reported threshold improvement is not claimed to generalise.

E. Smoothed surrogate rises during gradient descent

The smoothed surrogate cost rises from 0.2907 to 0.4399 over the recorded 29 gradient-descent updates. The grid winner (0.30, 0.45) has hard cost 0.2829, while the returned point (0.60, 0.75) has hard cost 0.3872. The returned point is therefore worse than the grid winner under hard cost. Projection onto the box and gap constraints does not guarantee a descent step. The baseline-to-returned-point reduction (0.3999 to 0.3872, about 3.16%) must not be attributed to gradient descent; the grid search supplied the lower-cost starting point.

F. Regulariser-tied thresholds

The cost includes the regulariser $0.05[(t_r - 0.35)^2 + (t_b - 0.70)^2]$, which pulls thresholds toward (0.35, 0.70). This makes the baseline thresholds the regulariser target rather than an independently justified universal standard. A different target would change the objective and could change the selected thresholds. The baseline comparison should therefore be interpreted as a comparison against this specific regularised baseline.

Additional interpretation: the returned point is boundary-clipped and non-stationary; KKT multipliers were not verified. The Hessian diagnostic alone does not establish constrained optimality. The threshold experiment is a useful numerical case study, but it does not demonstrate successful descent or out-of-sample threshold generalisation.

X. CONCLUSION

ARES demonstrates an offline QR-code URL inspection pipeline and a cost-sensitive two-threshold decision policy. The classifier's recorded ranking and calibration metrics are strong on the constructed, domain-grouped evaluation split, but the split has high phishing prevalence and the data and features impose important limits on deployment claims. The calculus experiment provides a sigmoid-smoothed objective and numerical derivative diagnostics. However, the recorded gradient-descent path increases the surrogate cost, the returned point is not stationary, and its hard cost is higher than the grid winner's. The reported 3.16% hard-cost reduction relative to the baseline should therefore be described carefully and not credited to gradient descent. Future work should separate threshold tuning from final evaluation, broaden benign URL diversity, evaluate realistic prevalence, and revise the constrained optimisation procedure with a verified descent method and explicit constrained optimality checks.

ACKNOWLEDGMENT

The authors acknowledge the guidance of the faculty and the support of RV University.

REFERENCES

[1] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[2] A. Niculescu-Mizil and R. Caruana, "Predicting Good Probabilities with Supervised Learning," in *Proc. 22nd Int. Conf. Machine Learning (ICML)*, 2005, pp. 625–632.

[3] J. Nocedal and S. J. Wright, *Numerical Optimization*, 2nd ed. New York, NY, USA: Springer, 2006.

[4] S. Boyd and L. Vandenberghe, *Convex Optimization*. Cambridge, U.K.: Cambridge University Press, 2004.

APPENDIX A. CORE THRESHOLD ROUTINES AND ANALYTIC GRADIENT DERIVATION

The submitted `calculus_threshold.py` contains the hard two-threshold rates, smoothed rates, hard and smoothed expected-cost functions, central finite-difference gradient, finite-difference Hessian, grid search, projected updates, and final-point diagnostics. The report summarises these routines rather than reproducing an unverified or altered code listing.

A.1 Sigmoid derivative

Let $\sigma(z) = 1/(1 + e^{-z})$. Differentiating gives

$$\sigma'(z) = \frac{e^{-z}}{(1+e^{-z})^2} = \sigma(z)[1 - \sigma(z)]. \tag{14}$$

A.2 Chain rule

For $s_r(p) = \sigma\big((p - t_r)/\tau\big)$, $\partial s_r / \partial t_r = -s_r(1 - s_r)/\tau$. For $s_b(p) = \sigma\big((p - t_b)/\tau\big)$, $\partial s_b / \partial t_b = -s_b(1 - s_b)/\tau$. The

cross derivatives are zero because each sigmoid depends on only its own threshold.

A.3 Derivatives of smoothed rates

$\widetilde{\mathrm{FPR}} = (1/N_b)\sum_{y_i=0} s_r\,(p_i)$, so

$$\frac{\partial \widetilde{\mathrm{FPR}}}{\partial t_r} = -\frac{1}{N_b \tau}\sum_{y_i=0} s_r\,(p_i)\big(1 - s_r(p_i)\big). \tag{15}$$

$\widetilde{\mathrm{FNR}} = 1 - \big(1/N_p\big)\sum_{y_i=1} s_b\,(p_i)$, so

$$\frac{\partial \widetilde{\mathrm{FNR}}}{\partial t_b} = \frac{1}{N_p \tau}\sum_{y_i=1} s_b\,(p_i)\big(1 - s_b(p_i)\big). \tag{16}$$

$\widetilde{\mathrm{ReviewRate}} = (1/N)\sum_i s_r\,(p_i)\big(1 - s_b(p_i)\big)$. By the product rule,

$$\begin{aligned} \frac{\partial \widetilde{\mathrm{ReviewRate}}}{\partial t_r} &= -\frac{1}{N\tau}\sum_i s_r\,(p_i)\big(1 - s_r(p_i)\big)\big(1 - s_b(p_i)\big), \text{ and} \\ \frac{\partial \widetilde{\mathrm{ReviewRate}}}{\partial t_b} &= \frac{1}{N\tau}\sum_i s_r\,(p_i) s_b(p_i)\big(1 - s_b(p_i)\big). \end{aligned} \tag{17}$$

A.4 Full symbolic gradient

With $R = \lambda[(t_r - t_r^0)^2 + (t_b - t_b^0)^2]$, the regulariser derivatives are $2\lambda(t_r - t_r^0)$ and $2\lambda(t_b - t_b^0)$. Therefore:

$$\begin{aligned} \frac{\partial \tilde{J}}{\partial t_r} = \; & -\frac{c_{FP}}{N_b \tau}\sum_{y_i=0} s_r\,(1 - s_r) \\ & -\frac{c_{review}}{N\tau}\sum_i s_r\,(1 - s_r)(1 - s_b) \\ & +2\lambda(t_r - t_r^0). \end{aligned} \tag{18}$$

$$\begin{aligned} \frac{\partial \tilde{J}}{\partial t_b} = \; & \frac{c_{FN}}{N_p \tau}\sum_{y_i=1} s_b\,(1 - s_b) \\ & +\frac{c_{review}}{N\tau}\sum_i s_r\, s_b(1 - s_b) \\ & +2\lambda(t_b - t_b^0). \end{aligned} \tag{19}$$

Implementation distinction: `compute_cost_gradient_fd` evaluates central finite differences with $\epsilon = 10^{-4}$; the analytic expressions above are a reference derivation and are not directly evaluated by that function. The implementation also computes second derivatives by finite differences.

A.5 Core Code Excerpts

The two excerpts below show the central finite-difference gradient and the projected update that are used in the reported experiment; the full source file is calculus_threshold.py in the submitted repository.

```
def compute_cost_gradient_fd(y_true, y_proba, t_review, t_block,
                             cost_fp=1.0, cost_fn=5.0,
                             cost_review=0.5, eps=1e-4, tau=0.05):
    def c(r, b):
        return expected_cost_smoothed(
            y_true, y_proba, r, b,
            cost_fp, cost_fn, cost_review, tau)
    dC_d_review = (c(t_review + eps, t_block) -
                   c(t_review - eps, t_block)) / (2 * eps)
    dC_d_block = (c(t_review, t_block + eps) -
                  c(t_review, t_block - eps)) / (2 * eps)
    return {"dC_d_review": dC_d_review, "dC_d_block": dC_d_block}
```

Listing 1. Central finite-difference gradient (compute_cost_gradient_fd).

```
for k in range(max_iter):
    grad = compute_cost_gradient_fd(...)
    tr_new = float(np.clip(tr - lr * grad["dC_d_review"], 0.15, 0.60))
    tb_new = float(np.clip(tb - lr * grad["dC_d_block"], 0.45, 0.95))
    if tb_new - tr_new < 0.15:
        tb_new = float(np.clip(tr_new + 0.15, 0.45, 0.95))
```

Listing 2. Projected gradient descent update.

These excerpts are the two code paths that produced the results reported in Section VIII; the surrounding grid search, cost function, and Hessian diagnostic are summarised in Sections II–V and not reproduced here.

APPENDIX B. NUMERICAL TRACEABILITY AUDIT

The following table records the numerical values used in the report and their canonical artefact fields. Values quoted to fewer decimal places in prose are rounded displays of the source values.

TABLE VII

NUMERICAL TRACEABILITY AUDIT

Reported quantity	Value	Traceability
Raw rows / benign / phishing	160064 / 820 / 159244	baseline.test.raw_dataset_summary.total_rows / benign_rows / phishing_rows
Train size / train domains	17057 / 5985	baseline.test.n_train / n_train_domains
Test size / test domains	5083 / 1497	baseline.test.n_test / n_test_domains
Test phishing prevalence	0.7478	baseline.test.test_phishing_prevalence
Random seed	42	baseline.test.random_seed
Software versions	sklearn 1.9.1; NumPy 2.5.4; Python	baseline.test.sklearn_version / numpy_version / python_version

Reported quantity	Value	Traceability
	3.14.2	
ROC AUC / PR AUC / Brier	0.9931 / 0.9977 / 0.0302	baseline.test.roc_auc / pr_auc / brier
Precision / F1 / TPR / FPR	0.9833 / 0.9707 / 0.9584 / 0.0484	baseline.test.precision / f1_score / tpr / fpr
Classifier confusion matrix	[[1220,62],[158,3643]]	baseline.test.confusion_matrix
Precision at prevalence 0.5 / 0.05 / 0.01 / 0.001	0.952 / 0.5105 / 0.1668 / 0.0195	baseline.test.precision_at_prevalence
Baseline thresholds	0.35 / 0.70	calculus_experiment.baseline.review / block
Baseline hard cost	0.3998725901990573	calculus_experiment.baseline.cost
Baseline FPR / FNR / review	0.0694227769 / 0.0623520126 / 0.0373795003	calculus_experiment.baseline.rates
Baseline policy counts	62,27,109,128,1193,3564	calculus_experiment.baseline.rates.fp_review / fp_block / fn_allow / fn_review / tn / tp
Returned thresholds	0.60 / 0.75	calculus_experiment.optimized.review_threshold / block_threshold
Returned hard cost	0.387223 (rounded); 0.38722259534246795 in results brief	calculus_experiment.optimized.optimal_cost / final_cost_hard
Returned smoothed cost	0.439942	calculus_experiment.optimized.optimal_cost_smoothed / final_cost
Returned FPR / FNR / review	0.0343 / 0.0681 / 0.0179	calculus_experiment.optimized.rates_at_optimum
Returned policy counts	25,19,193,66,1238,3542	calculus_experiment.optimized.rates_at_optimum.fp_review / fp_block / fn_allow / fn_review / tn / tp
Gradient components	−0.153059 / 1.621676	optimized.derivative_check.first_derivative_review / first_ derivative_block
Hessian entries	0.405678 / 17.086192 / −0.08807	optimized.derivative_check.second_derivative_review / second_ derivative_block / cross_derivative_review_block
Hessian determinant / trace	6.923728 / 17.491870	optimized.derivative_check.hessian_det; trace computed as sum of diagonal entries
Active constraints	review_box_0.60; gap_0.15	optimized.derivative_check.boundary_hit
Bayes reference threshold	0.1667	optimized.derivative_check.bayes_optimal_single_threshold
Gradient steps / history entries	29 / 30	optimized.derivative_check.gradient_descent_iterations; len(optimized.history)
Learning rate / tau / grad tolerance	0.05 / 0.05 / 0.01	optimized.derivative_check.learning_rate / temperature_tau / grad_tol
Grid winner thresholds / costs	(0.30,0.45); smoothed 0.2907196813; hard 0.2829383160	results_brief.md grid-winner computation; not a field in calculus_ experiment.json
Finite difference epsilon	0.0001	calculus_threshold.py compute_cost_gradient_fd default eps
Regulariser coefficient / target	0.05 / (0.35,0.70)	calculus_threshold.py expected_cost and expected_cost_smoothed lines 41,46
Limitations 1, 2 and 6 evidence	820 base domains; 24 features; default thresholds	User-supplied evidence statements; train_model.py / feature_ extractor.py / adaptive_agent.py not attached for independent line verification
Submission date	1 November 2026	Project conversation deadline context; confirm if institutional cover date differs

Traceability cautions: (i) The grid-winner hard cost is sourced from the supplied results brief rather than a field in calculus_experiment.json. (ii) The 24-feature count and source-function descriptions are supplied project evidence; the corresponding feature_extractor.py and train_model.py files were not attached in this report-generation batch. (iii) No held-out threshold-tuning experiment or 1,017-URL result is included or claimed.